Perimity — Database Design Reference

Smart Campus Access & Gate Pass Management System Microservices Architecture · Spring Boot · PostgreSQL + MongoDB · AWS

Overview

Perimity uses a **database-per-service** pattern. Each microservice owns its own database — no shared database between services. Transactional data lives in PostgreSQL; high-volume append-only logs live in MongoDB.

Service	Database	Type	Port
Auth Service	AuthDB	PostgreSQL	5432
User Service	UserDB	PostgreSQL	5433
Gate Pass Service	GatePassDB	PostgreSQL	5434
Campus Service	CampusDB	PostgreSQL	5435
Guard Service	EntryLogDB	MongoDB	27017
QR Service	QRDB	PostgreSQL	5436

AuthDB (PostgreSQL)

Owns authentication, users, and security audit trail.

Table	What it stores	Why
`users`	Email, password hash, role, campus_id	Every person who logs in — one record per user across all campuses
`otp_verifications`	SHA-256 hashed OTP, expiry, attempts	Visitor email verification before form submission

Table	What it stores	Why
`audit_logs`	Who did what, when, from where	Security trail — admin actions, logins, approvals

Notes

- Passwords stored as bcrypt hashes, never plain text.
- OTP stored as SHA-256 hash, never plain text; expires in 10 minutes; locked after 5 attempts.
- `campus_id` scopes each user to their institution (multi-tenant).

UserDB (PostgreSQL)

Owns student and faculty profile information.

Table	What it stores	Why
`student_profiles`	Roll no, year, aadhaar, address, photo S3 key	Student identity info — photo file is on S3, only the key stored here
`faculty_profiles`	Employee ID, designation, qualification	Faculty identity — same pattern
`departments`	DAC, DBDA, DESD etc per campus	Each campus has its own department list
`documents`	S3 key, file name, mime type, verified flag	Aadhaar uploads, certificates — actual files on S3

Notes

- No `semester` field anywhere — CDAC Mumbai has no semester concept.
- Actual files (photos, documents) live on S3; the database stores only the S3 key.
- `user_id` references the user in AuthDB (cross-service reference by convention, not a DB-enforced foreign key).

GatePassDB (PostgreSQL)

Owns the visitor request workflow and gate pass lifecycle. This is the core business logic.

Table	What it stores	Why
`visitor_requests`	Visitor name, email, purpose, host, dates, OTP status, approval status	Full visitor registration form data before a pass is issued
`gate_passes`	Pass holder, campus, valid dates, status (Active/Expired/Revoked), S3 keys for QR + PDF	The actual gate pass record — links to the QR image and PDF on S3
`bulk_upload_batches`	Excel file S3 key, row counts, processing status	Tracks faculty bulk imports — how many rows valid, invalid, processed

Notes

- A `visitor_request` gets approved, then becomes a `gate_pass`.
- Pass status transitions: Pending → Active → Expired / Revoked.
- Bulk upload lets faculty create many passes at once from an Excel sheet.

CampusDB (PostgreSQL)

Owns multi-tenant campus management.

Table	What it stores	Why
`campuses`	Name, code, address, logo S3 key, admin user	One row per institution — CDAC Mumbai, CDAC Pune etc
`campus_gates`	Gate name, location per campus	Main Gate, Back Gate etc — guard scans happen at a specific gate
`campus_config`	Key-value settings per campus	Approval required? Re-entry allowed? Each campus has own rules

Notes

- `campus_config` is a key-value store so each campus can have custom settings without schema changes.
- A single deployment serves any number of campuses.

EntryLogDB (MongoDB)

Owns high-volume, append-only gate scan events.

Collection	What it stores	Why
`entry_logs`	Scan result, pass id, guard id, timestamp, gate, device	Every QR scan creates one document — allow or deny both recorded

Why MongoDB here

- Write-heavy — every scan is an insert.
- No joins needed — flat document per scan.
- High volume — millions of rows over time.
- Shardable by `campus_id` as volume grows.
- Queried mostly by simple filters (campus, date range, pass holder).

Indexes

- Compound: `campus_id + scanned_at` (covers most queries)
- Single: `pass_id` , `holder_user_id`
- Optional TTL on `scanned_at` for auto-expiry of old logs

QRDB (PostgreSQL)

Owns pass tokens and QR/PDF generation jobs.

Table	What it stores	Why
`qr_records`	Token hash, pass id, S3 keys for QR + PDF, valid dates, is_active	The validated pass token — guard checks this when scanning
`generation_jobs`	Job status, retry count, error message	Tracks async QR generation jobs from RabbitMQ queue

Notes

- QR token is AES-256 encrypted (pass_id + campus_id + expiry), never plain text.
- Failed generation jobs are retried a bounded number of times before manual escalation.

How QR and PDF Get Stored — Step by Step

1. Admin approves a visitor request or student profile.
2. Gate Pass Service creates a `gate_pass` record (status = Pending, no QR yet).
3. Gate Pass Service drops a job into the RabbitMQ queue: `{ pass_id, holder_name, campus, valid_from, valid_to }`.
4. QR/PDF Service (Python) picks up the job.
5. Python generates:
    - A unique signed token (AES-256 encrypted pass_id + campus_id + expiry)
    - A QR code image PNG (from the token)
    - A PDF pass (QR image + name + photo + validity dates)
6. Python uploads both files to AWS S3:
    - `s3://perimity-passes/campus-1/passes/pass-123-qr.png`
    - `s3://perimity-passes/campus-1/passes/pass-123.pdf`
7. Python stores the S3 keys + token hash in QRDB.
8. Gate Pass Service updates the `gate_pass` record:
    - `qr_s3_key = "campus-1/passes/pass-123-qr.png"`
    - `pdf_s3_key = "campus-1/passes/pass-123.pdf"`
    - `status = Active`
9. Notification Service emails the PDF to the holder.
10. Guard scans QR at the gate:
    - Guard Service decrypts the token
    - Checks QRDB: is the token valid? is the pass active? is the date in range?
    - Records the result in MongoDB `entry_logs`
    - Returns GREEN / RED / AMBER to the scanner UI

S3 Folder Structure

```
perimity-bucket/
├── campuses/
│   └── campus-1/
│       └── logo.png
├── profiles/
│   └── user-42/
│       ├── photo.jpg
```

```
│       └── aadhaar.pdf
└── passes/
    └── campus-1/
        ├── pass-123-qr.png   ← QR image
        └── pass-123.pdf      ← printable PDF pass
```

Where Everything Is Stored — Quick Reference

Thing	Where stored
User passwords	AuthDB — bcrypt hashed
OTP codes	AuthDB — SHA-256 hashed, never plain
Profile photos	S3 — only S3 key in UserDB
Aadhaar documents	S3 — only S3 key in UserDB
QR code image	S3 — S3 key in GatePassDB + QRDB
PDF pass	S3 — S3 key in GatePassDB + QRDB
QR token	QRDB — AES-256 encrypted, never plain
Gate scan events	MongoDB — one document per scan
Everything else	PostgreSQL

Key Design Principles

- **Database per service** — no service reads another service's database directly; they call each other's APIs.
- **Files on S3, keys in DB** — never store binary files in the database.
- **Secrets always hashed** — passwords (bcrypt), OTPs (SHA-256), QR tokens (AES-256).
- **Multi-tenant by campus_id** — every campus-specific row carries a `campus_id`.
- **PostgreSQL for transactions, MongoDB for logs** — strong consistency where it matters, scale where volume grows.